CPSC-354 Report

Mitchell Toney
Chapman University

December 14, 2025

Abstract

This report documents coursework from CPSC-354, covering foundational topics in programming languages and computation theory. Topics include abstract rewriting systems, termination proofs, lambda calculus and Church encodings, fixed-point combinators, formal grammars and parse trees, theorem proving in Lean, propositional logic, and interpreter implementation. Each week builds toward understanding how programming languages are formally defined, evaluated, and reasoned about.

Contents

1 Introduction

2 Week by Week

2.1 Week 1: HW 1 - MU Puzzle

The MU puzzle is a puzzle created by Douglas Hofstadter. It consists of four rules that can be applied to a string MI.

1. $xI \to xIU$
2. $Mx \to Mxx$
3. $xIIIy \to xUy$
4. $xUUy \to xy$

When first approaching this puzzle, the first strategy that came to mind was to take advantage of rule number 2 to keep duplicating the I's until there is a multiple of three, then using rules 3 and 4 to get rid of the I's and leave a remaining U.

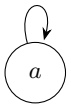
The issue with this is that $2^n \bmod 3$ will never equal 0, it infinitely cycles between equaling 1 and 2, and without being able to get rid of all the I's, which would require them being a multiple of 3, you will never be able to get MU.

Thus, the puzzle is not solvable.

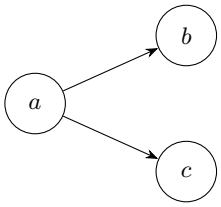
2.2 Week 2: HW 2 - Abstract Rewriting Systems

1. $A = \varnothing,\ R = \varnothing$

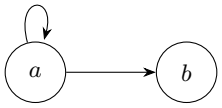
2. $A = \{a\},\ R = \varnothing$



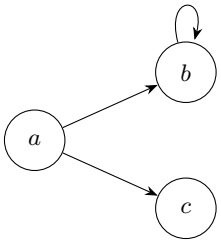
3. $A = \{a\},\ R = \{(a, a)\}$



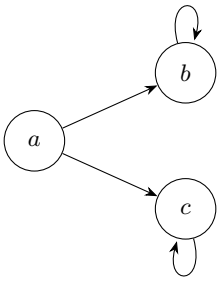
4. $A = \{a, b, c\}$, $R = \{(a, b), (a, c)\}$



5. $A = \{a, b\}$, $R = \{(a, a), (a, b)\}$



6. $A = \{a, b, c\}$, $R = \{(a, b), (b, b), (a, c)\}$



7. $A = \{a, b, c\}$, $R = \{(a, b), (b, b), (a, c), (c, c)\}$

#	Terminating	Confluent	Unique NFs
1	Yes	Yes	Yes
2	Yes	Yes	Yes
3	No	Yes	No
4	Yes	No	No
5	No	Yes	Yes
6	No	No	No
7	No	No	No

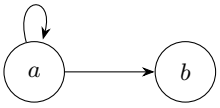
Confluent True, Terminating True, Unique NFs True

$A = \{a\},\ R = \varnothing$

Confluent True, Terminating True, Unique NFs False

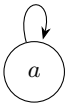
(no ARS exists)

Confluent True, Terminating False, Unique NFs True



$A = \{a, b\},\ R = \{(a, a), (a, b)\}$

Confluent True, Terminating False, Unique NFs False



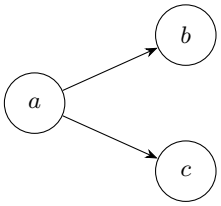
$A = \{a\},\ R = \{(a, a)\}$

Confluent False, Terminating True, Unique NFs True

impossible

(no ARS exists)

Confluent False, Terminating True, Unique NFs False



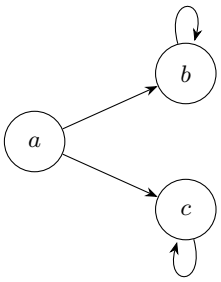
$A = \{a, b, c\},\ R = \{(a, b), (a, c)\}$

Confluent False, Terminating False, Unique NFs True

impossible

(no ARS exists)

Confluent False, Terminating False, Unique NFs False



$A = \{a, b, c\},\ R = \{(a, b), (a, c), (b, b), (c, c)\}$

2.3 Week 3: HW 3 - ARS Equivalence Classes

2.3.1 Exercise 5

Consider rewrite rules:

$$\begin{aligned} ab &\to ba \\ ba &\to ab \\ aa &\to \\ b &\to \end{aligned}$$

Example Reductions

Reducing `abba`:

$$\begin{aligned} abba &\to baba \quad (\text{using } ab \to ba) \\ baba &\to bbaa \quad (\text{using } ba \to ab) \\ bbaa &\to baa \quad (\text{using } b \to \varepsilon) \\ baa &\to aba \quad (\text{using } ba \to ab) \\ aba &\to baa \quad (\text{using } ab \to ba) \end{aligned}$$

There is an infinite loop between `aba` and `baa`.

Reducing `bababa`:

$$\begin{aligned} bababa &\to ababab \quad (\text{using } ba \to ab) \\ ababab &\to baabab \quad (\text{using } ab \to ba) \\ baabab &\to ababab \quad (\text{using } ba \to ab) \end{aligned}$$

This is an infinite loop between `ababab` and `baabab`.

Why the ARS is not terminating The ARS is not terminating because the rules $ab \to ba$ and $ba \to ab$ create infinite cycles. These rules allow us to swap adjacent a and b characters indefinitely, leading to non-terminating reduction sequences.

Non-equivalent strings Two strings that are not equivalent: `a` and `aa`.

The string `a` cannot be reduced further, while `aa` reduces to nothing using the rule $aa \to$. Since $\mathit{nothing} \neq a$, these strings are in different equivalence classes.

Equivalence classes The equivalence relation $\leftrightarrow^*$ has infinitely many equivalence classes. Each equivalence class can be characterized by the number of a's modulo 2 and the number of b's modulo 1.

The equivalence classes are:

- $[\varepsilon]$: strings with even number of a's and no b's
- $[a]$: strings with odd number of a's and no b's
- $[b]$: strings with any number of a's and at least one b

The normal forms are: ε, a, and b.

Modifying the ARS to be terminating To make the ARS terminating without changing equivalence classes, we can remove the symmetric rules and keep only one direction:

$$\begin{aligned} ba &\to ab \\ aa &\to \varepsilon \\ b &\to \varepsilon \end{aligned}$$

This eliminates the infinite cycles while preserving the same equivalence relation.

2.3.2 Semantic question

Parity of a's: "Does this string contain an odd number of a's?"
Answer: Yes if the normal form is a, No if the normal form is ε.

Exercise 5b

Consider rewrite rules:

$$\begin{aligned} ab &\to ba \\ ba &\to ab \\ aa &\to a \\ b &\to \end{aligned}$$

Reducing `abba`:

$$\begin{aligned} abba &\to baba \\ baba &\to abba \end{aligned}$$

Reducing `bababa`:

$$\begin{aligned} bababa &\to ababab \\ ababab &\to bababa \end{aligned}$$

Why not terminating. The symmetric swaps $ab \leftrightarrow ba$ allow infinite rewriting.

Non-equivalent strings. a and ε are not equivalent.

Equivalence classes. Exactly two: $[\varepsilon]$ (no a's; all b's delete) and $[a]$ (at least one a; since $aa \sim a$). Normal forms (under a terminating orientation): ε and a.

Terminating variant.

$$ba \to ab$$
$$aa \to a$$
$$b \to$$

This gives a complete semantics to the ARS: it computes the invariant for any input string.

2.4 Week 4: HW 4 - Termination Proofs

2.4.1 HW4.1: Termination Proof for GCD

Consider the following algorithm:

```
while b != 0:
    temp = b
    b = a mod b
    a = temp
return a
```

Assume: Work over integers with Euclidean division: inputs $a \in \mathbb{Z}$, $b \in \mathbb{N}$; if $b \neq 0$ then $0 \leq a \bmod b < b$.

Model: States $A = \mathbb{Z} \times \mathbb{N}$. One step $(a, b) \to (a', b')$ is one loop iteration.

Measure: $\phi : A \to \mathbb{N},\ \phi(a, b) = b$.

Show: $(a, b) \to (a', b') \Rightarrow \phi(a', b') < \phi(a, b)$.

If $b \neq 0$, the update sets $b' = a \bmod b$ with $0 \leq b' < b$; hence $\phi(a', b') = b' < b = \phi(a, b)$.

ϕ strictly decreases in $\mathbb{N}$, so there is no infinite $\to$-chain; eventually $b = 0$ and the loop stops. Thus the algorithm terminates under the stated conditions.

2.4.2 HW4.2: Termination Proof for Merge Sort

Consider the following fragment of an implementation of merge sort:

```
function merge_sort(arr, left, right):
    if left >= right:
        return
    mid = (left + right) / 2
    merge_sort(arr, left, mid)
    merge_sort(arr, mid+1, right)
    merge(arr, left, mid, right)
```

Prove that

$\phi(\texttt{left}, \texttt{right}) = \texttt{right} - \texttt{left} + 1$

is a measure function for merge_sort.

Show: For

```
merge_sort(arr, left, right):
  if left >= right: return
    mid = floor((left + right)/2)
  merge_sort(arr, left, mid)
  merge_sort(arr, mid+1, right)
  merge(arr, left, mid, right)
```

the function $\phi(\text{left}, \text{right}) = \text{right} - \text{left} + 1$ is a strictly decreasing measure, hence `merge_sort` terminates.

Assume: `left,right`$\in \mathbb{Z}$ with $0 \leq \text{left} \leq \text{right} < |arr|$. Division for `mid` is integer. If `left < right`, then $\text{left} \leq \text{mid} < \text{right}$. `merge` makes no recursive calls.

Model: States are intervals $A = \{(l, r) \in \mathbb{Z}^2 \mid l \leq r\}$. A "step" is a recursive edge from (l, r) (with $l < r$) to each child (l, mid) and $(\text{mid} + 1, r)$.

Measure: $\phi : A \to \mathbb{N}, \quad \phi(l, r) = r - l + 1$.

Let $l < r$ and $\text{mid} = \lfloor (l + r)/2 \rfloor$ so $l \leq \text{mid} < r$.

First child: $\phi(l, \text{mid}) = \text{mid} - l + 1 \leq (r - 1) - l + 1 = r - l = \phi(l, r) - 1 < \phi(l, r)$.

Second child: $\phi(\text{mid} + 1, r) = r - \text{mid} \leq r - l = \phi(l, r) - 1 < \phi(l, r)$.

Every recursive edge strictly decreases ϕ in $\mathbb{N}$, which is well-founded. Thus no infinite recursion is possible; the calls bottom out at states with $\phi \in \{0, 1\}$ (i.e., `left` $\geq$ `right`), where the function returns. Therefore $\phi(\text{left}, \text{right}) = \text{right} - \text{left} + 1$ is a valid measure and `merge_sort` terminates under the stated conditions.

2.5 Week 5: HW 5 - Lambda Calculus Evaluation

2.5.1 Lambda Calculus Workout Evaluation (Corrected)

Evaluate: $(\lambda f.\lambda x.\, f(f\, x))(\lambda g.\lambda y.\, g(g(g\, y)))$

Use α-renaming to avoid capture.

$$
\begin{aligned}
(\lambda f.\lambda x.\, f(f\, x))(\lambda g.\lambda y.\, g(g(g\, y))) &\to \lambda x.\, (\lambda g.\lambda y.\, g(g(g\, y)))\big((\lambda g.\lambda y.\, g(g(g\, y)))\, x\big) \\
(\lambda g.\lambda y.\, g(g(g\, y)))\, x &\to \lambda y.\, x(x(x\, y)) \quad (\alpha\text{-rename inner } y) \\
\Rightarrow\ \lambda x.\, (\lambda g.\lambda y.\, g(g(g\, y)))\, (\lambda y.\, x(x(x\, y))) &\to \lambda x.\lambda y.\, x^9 y.
\end{aligned}
$$

Hence the normal form is $\lambda f.\lambda x.\, f^9 x$ (Church numeral 9).

2.6 Week 6: HW 6 - Fixed Point Combinators

2.6.1 Fixed Point Combinator Exercise

Compute `fact 3` following the computation rules for `fix`, `let`, and `let rec`:

Given:

$$E_0 \;=\; \mathsf{let\ rec\ fact} = \lambda n.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * \mathsf{fact}\,(n-1)\ \mathsf{in\ fact}\ 3$$

Abbreviations:

$$\mathsf{F} \stackrel{\text{def}}{=} \lambda f.\, \lambda n.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * f\,(n-1) \qquad \mathsf{FACT} \stackrel{\text{def}}{=} \mathsf{fix}\,\mathsf{F}$$

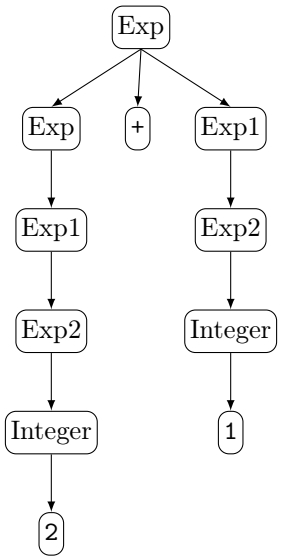
Computation:

$$
\begin{aligned}
E_0 &\xrightarrow{\text{def of let rec}} \mathsf{let}\ \mathsf{fact} = \mathsf{fix}(\lambda f.\lambda n.\mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * f\,(n-1))\ \mathsf{in}\ \mathsf{fact}\,3 \\
&\xrightarrow{\text{def of let}} (\lambda \mathsf{fact}.\ \mathsf{fact}\,3)\ \mathsf{fix}(\lambda f.\lambda n.\mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * f\,(n-1)) \\
&\xrightarrow{\beta} \big(\mathsf{fix}(\lambda f.\lambda n.\mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * f\,(n-1))\big)\ 3 \\
&\equiv \mathsf{FACT}\,3 \\
&\xrightarrow{\text{def of fix}} (\mathsf{F}\,\mathsf{FACT})\ 3 \\
&\xrightarrow{\beta} (\lambda n.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * \mathsf{FACT}\,(n-1))\ 3 \\
&\xrightarrow{\beta} \mathsf{if}\ 3 = 0\ \mathsf{then}\ 1\ \mathsf{else}\ 3 * \mathsf{FACT}\,(3-1) \\
&\xrightarrow{\text{def of if}} 3 * \mathsf{FACT}\,(3-1) \\
&\xrightarrow{\text{arith}} 3 * \mathsf{FACT}\,2 \\
&\xrightarrow{\text{def of fix}} 3 * (\mathsf{F}\,\mathsf{FACT})\ 2 \\
&\xrightarrow{\beta} 3 * (\lambda n.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * \mathsf{FACT}\,(n-1))\ 2 \\
&\xrightarrow{\beta} 3 * \mathsf{if}\ 2 = 0\ \mathsf{then}\ 1\ \mathsf{else}\ 2 * \mathsf{FACT}\,(2-1) \\
&\xrightarrow{\text{def of if}} 3 * (2 * \mathsf{FACT}\,(2-1)) \\
&\xrightarrow{\text{arith}} 3 * (2 * \mathsf{FACT}\,1) \\
&\xrightarrow{\text{def of fix}} 3 * \big(2 * (\mathsf{F}\,\mathsf{FACT})\ 1\big) \\
&\xrightarrow{\beta} 3 * \big(2 * (\lambda n.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * \mathsf{FACT}\,(n-1))\ 1\big) \\
&\xrightarrow{\beta} 3 * \big(2 * \mathsf{if}\ 1 = 0\ \mathsf{then}\ 1\ \mathsf{else}\ 1 * \mathsf{FACT}\,(1-1)\big) \\
&\xrightarrow{\text{def of if}} 3 * \big(2 * (1 * \mathsf{FACT}\,(1-1))\big) \\
&\xrightarrow{\text{arith}} 3 * \big(2 * (1 * \mathsf{FACT}\,0)\big) \\
&\xrightarrow{\text{def of fix}} 3 * \big(2 * (1 * (\mathsf{F}\,\mathsf{FACT})\ 0)\big) \\
&\xrightarrow{\beta} 3 * \big(2 * (1 * (\lambda n.\ \mathsf{if}\ n = 0\ \mathsf{then}\ 1\ \mathsf{else}\ n * \mathsf{FACT}\,(n-1))\ 0)\big) \\
&\xrightarrow{\beta} 3 * \big(2 * (1 * \mathsf{if}\ 0 = 0\ \mathsf{then}\ 1\ \mathsf{else}\ 0 * \mathsf{FACT}\,(0-1))\big) \\
&\xrightarrow{\text{def of if}} 3 * \big(2 * (1 * 1)\big) \\
&\xrightarrow{\text{arith}} 3 * (2 * 1) \xrightarrow{\text{arith}} 3 * 2 \xrightarrow{\text{arith}} 6\,.
\end{aligned}
$$

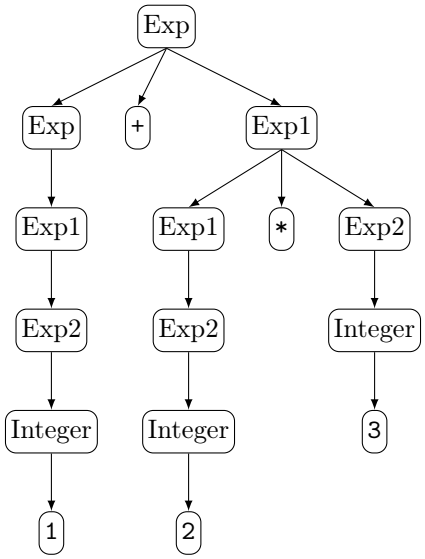
Result: `fact 3 = 6`

2.7 Week 7: HW 7 - Parse Trees Exercise

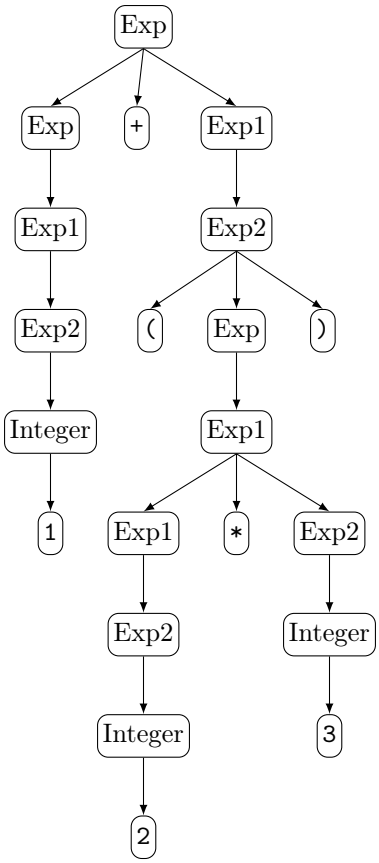
2.7.1 String: $2 + 1$



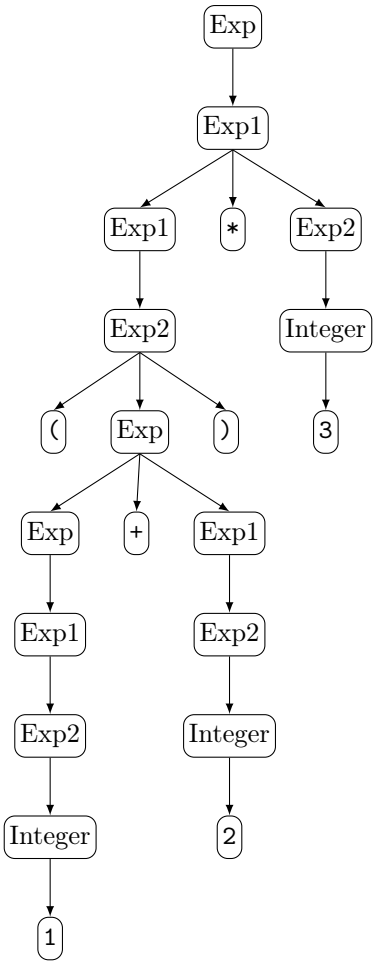
2.7.2 String: $1 + 2 * 3$



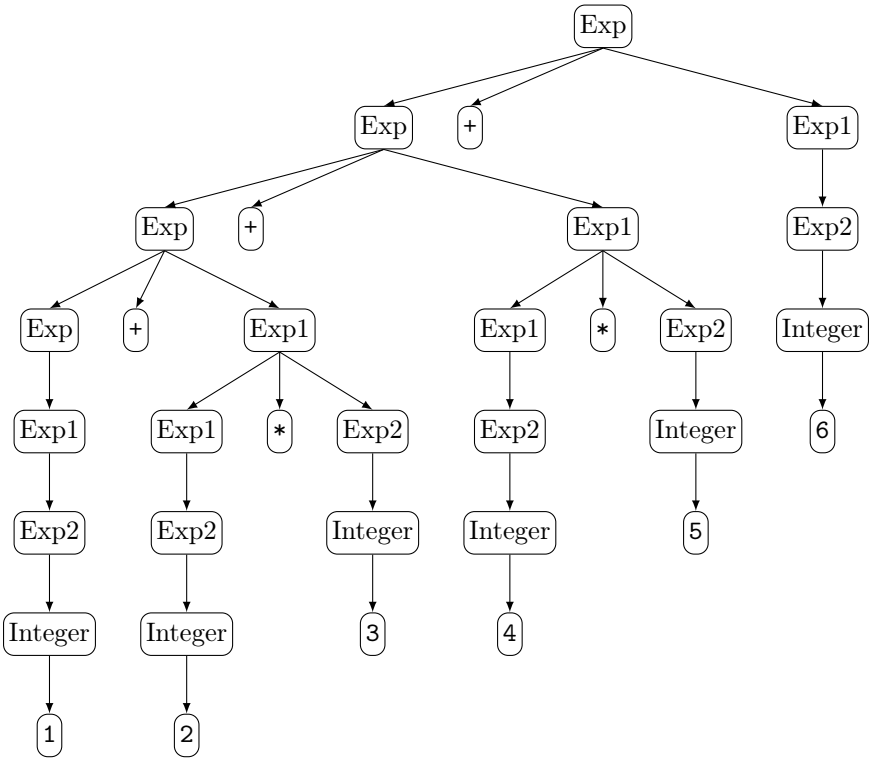
2.7.3 String: $1 + (2 * 3)$



2.7.4 String: $(1 + 2) * 3$



2.7.5 String: $1 + 2 * 3 + 4 * 5 + 6$



2.8 Week 8: HW 8 - Lean Proofs

2.8.1 Exercise 1

Level 5:

$a + (b + 0) + (c + 0) = a + b + c$

```
rw [add_zero]
rw [add_zero]
rfl
```

Level 6:

$a + (b + 0) + (c + 0) = a + b + c$

```
rw [add_zero c]
rw [add_zero b]
rfl
```

Level 7:

$succn = n + 1$

```
rw [one_eq_succ_zero]
rw [add_succ]
rw[add_zero]
rfl
```

Level 8:

$2 + 2 = 4$

```
rw [two_eq_succ_one]
rw [four_eq_succ_three]
rw [three_eq_succ_two]
rw [two_eq_succ_one]
rw [one_eq_succ_zero]
rw [add_succ]
rw [add_succ]
rw [add_zero]
rfl
```

Level 7 Natural Language Proof:

$$\operatorname{succ} n = n + 1$$

Replace 1 by $\operatorname{succ} 0$:

$$\operatorname{succ} n = n + \operatorname{succ} 0$$

Use the definition of addition with a successor:

$$\operatorname{succ} n = \operatorname{succ}(n + 0)$$

Simplify $n + 0$ to n:

$$\operatorname{succ} n = \operatorname{succ} n$$

Both sides are identical, so the equality holds.

2.9 Week 9: HW 9 - Induction Proofs

2.9.1 No Induction

```
rw [add_assoc a b c]
rw [add_comm b c]
rw [← add_assoc a c b]
```

For all $(a, b, c \in \mathbb{N})$,

$$a + b + c = a + c + b.$$

$$\begin{aligned} a + b + c &= a + (b + c) && \text{(associativity of } (+)) \\ &= a + (c + b) && \text{(commutativity of } (+)) \\ &= a + c + b && \text{(associativity of } (+)). \end{aligned}$$

2.9.2 Induction

```
induction c with d hd
rw [add_zero, add_zero]
rfl

rw[add_succ]
rw[add_succ]
rw[succ_add]
rw[hd]
```

For all $(a, b, c \in \mathbb{N})$,

$$a + b + c = a + c + b.$$

By induction on c.

Base case ($c = 0$).

$$\begin{aligned} a + b + 0 &= a + b && \text{(by } (x + 0 = x)\text{)} \\ &= a + 0 + b && \text{(by } (x + 0 = x)\text{)}, \end{aligned}$$

Assume $a + b + d = a + d + b$:

$$\begin{aligned} a + b + (d + 1) &= (a + b + d) + 1 && \text{(by } (x + (y + 1) = (x + y) + 1)\text{)} \\ &= (a + d + b) + 1 && \text{(induction hypothesis)} \\ &= (a + d + 1) + b && \text{(by } ((x + y) + 1 = (x + 1) + y)\text{)} \\ &= a + (d + 1) + b && \text{(associativity).} \end{aligned}$$

Thus $a + b + c = a + c + b$ for all $c \in \mathbb{N}$.

2.10 Week 10: HW 10 - Curry and Uncurry

2.10.1 Level 6: Curry

Objects: C, D, S : Prop

Assumptions: $h : C \wedge D \to S$

Goal:

$C \to D \to S$

Solution:

```
exact λ c d ↦ h (and_intro c d)
```

2.10.2 Level 7: Un-Curry

Objects: C, D, S : Prop

Assumptions: $h : C \to D \to S$

Goal:

$C \wedge D \to S$

Solution:

```
exact λ cd ↦ h (and_left cd) (and_right cd)
```

2.10.3 Level 8: Go buy chips and dip!

Objects: C, D, S : Prop

Assumptions: $h : (S \to C) \wedge (S \to D)$

Goal:

$S \to C \wedge D$

Solution:

```
exact λ s ↦ and_intro (and_left h s) (and_right h s)
```

2.10.4 Level 9: Uncertain Snacks

Objects: R, S : Prop

Goal:

$R \to (S \to R) \wedge (\neg S \to R)$

Solution:

```
exact λ r ↦ and_intro (λ s ↦ r) (λ _ ↦ r)
```

Question: Does currying versus uncurrying ever change definitional equality enough to affect the rewrites from the homework?

2.11 Week 11: HW 11 - Negation Proofs

2.11.1 Level 9: Allergy #1

Objects: P, A : Prop

Assumptions: $h : P \to \neg A$

Goal:

$\neg(P \wedge A)$

Solution:

```
exact λ pa : P ∧ A ↦ h pa.left pa.right
```

2.11.2 Level 10: Allergy #2

Objects: P, A : Prop

Assumptions: $h : \neg(P \wedge A)$

Goal:

$P \to \neg A$

Solution:

```
exact λ p : P ↦ λ a : A ↦ h (and_intro p a)
```

2.11.3 Level 11: Allergy: Triple Confusion

Objects: A : Prop

Assumptions: $h : \neg\neg\neg A$

Goal:

$\neg A$

Solution:

```
exact λ a : A ↦ h (λ na : ¬A ↦ na a)
```

2.11.4 Level 12

Objects: B, C : Prop

Assumptions: $h : \neg(B \to C)$

Goal:

$\neg\neg B$

Solution:

```
exact λ nb ↦ h (nb ≫ false_elim)
```

Question: How would the allergy scenario look different if explosion was rejected?

2.12 Week 12: HW 12 - Tower of Hanoi

2.12.1 Completed execution

Execution for `hanoi 5 0 2`:

```
hanoi 5 0 2
    hanoi 4 0 1
        hanoi 3 0 2
            hanoi 2 0 1
                hanoi 1 0 2 = move 0 2
                move 0 1
                hanoi 1 2 1 = move 2 1
            move 0 2
            hanoi 2 1 2
                hanoi 1 1 0 = move 1 0
                move 1 2
                hanoi 1 0 2 = move 0 2
        move 0 1
        hanoi 3 2 1
            hanoi 2 2 0
                hanoi 1 2 1 = move 2 1
                move 2 0
                hanoi 1 1 0 = move 1 0
            move 2 1
            hanoi 2 0 1
                hanoi 1 0 2 = move 0 2
                move 0 1
                hanoi 1 2 1 = move 2 1
    move 0 2
    hanoi 4 1 2
        hanoi 3 1 0
            hanoi 2 1 2
                hanoi 1 1 0 = move 1 0
                move 1 2
                hanoi 1 0 2 = move 0 2
            move 1 0
            hanoi 2 2 0
                hanoi 1 2 1 = move 2 1
                move 2 0
                hanoi 1 1 0 = move 1 0
```

```
move 1 2
hanoi 3 0 2
    hanoi 2 0 1
        hanoi 1 0 2 = move 0 2
        move 0 1
        hanoi 1 2 1 = move 2 1
    move 0 2
    hanoi 2 1 2
        hanoi 1 1 0 = move 1 0
        move 1 2
        hanoi 1 0 2 = move 0 2
```

2.12.2 Successful Moves

The sequence of moves for solving the Tower of Hanoi with 5 disks from peg 0 to peg 2:

```
0 → 2
0 → 1
2 → 1
0 → 2
1 → 0
1 → 2
0 → 2
0 → 1
2 → 1
2 → 0
1 → 0
2 → 1
0 → 2
0 → 1
2 → 1
0 → 2
1 → 0
1 → 2
0 → 2
1 → 0
2 → 1
2 → 0
1 → 0
1 → 2
0 → 2
0 → 1
2 → 1
0 → 2
1 → 0
1 → 2
0 → 2
```

2.12.3 Verirxefication of Moves

The moves listed above were tested on the Tower of Hanoi simulation website and confirmed to successfully find the solution for moving 5 disks from peg 0 to peg 2.

Question: Would the Tower of Hanoi problem still have a recursive definition if there were infinitely many disks?

2.13 Week 13: HW 13 - Lambda Calculus Interpreter

2.13.1 Exercise 2: Lambda Calculus Interpreter Testing

Lambda expressions from the lectures on Lambda Calculus and Church Encodings were added to `test.lc` and tested with the interpreter.

Test Results:

- `a b c d` → `(((a b) c) d)` (left associativity confirmed)
- `(a)` → `a` (parentheses reduction)
- `(λx.x) a` → `a` (beta-reduction working correctly)
- HW5 example: `(λf.λx.f(f(x))) (λf.λx.(f(f(f x))))` evaluates correctly

All tests confirmed the interpreter conforms to lambda calculus operational semantics.

2.13.2 Exercise 3: Capture-Avoiding Substitution

Investigation of capture-avoiding substitution through test cases and source code analysis.

Implementation:

The `substitute` function (lines 70-87) handles capture-avoidance by generating fresh variable names when substituting into lambda abstractions. When substituting into `λy.e` where the bound variable differs from the substitution target, the implementation:

- Generates a fresh name (Var1, Var2, etc.) via `NameGenerator`
- Alpha-renames the lambda abstraction
- Performs substitution in the renamed body

Test Case:

`(λy.x) [y/x]` → `(λVar1.y)` demonstrates that the free variable `x` is replaced by `y`, while the lambda is renamed to `Var1` to avoid variable capture. If the bound variable matches the substitution target, no substitution occurs (lines 78-79), preserving the binding.

2.13.3 Exercise 4: Normal Form Reduction

Investigation of whether all computations reduce to normal form.

Findings:

Not all computations reduce to normal form. The omega combinator `(λx.x x) (λx.x x)` causes infinite recursion (RecursionError), demonstrating that non-terminating expressions exist.

Evaluation Strategy:

The interpreter uses call-by-name evaluation (lines 37-53): it evaluates the left side of applications first, then substitutes without evaluating arguments.

- Terminating expressions reach normal form
- Non-terminating expressions loop indefinitely
- Expressions already in normal form (e.g., `x y` where variables are free) remain unchanged

2.13.4 Exercise 5: Minimal Non-Terminating Expression

Search for the smallest lambda expression that does not reduce to normal form.

Minimal Working Example:

The omega combinator `(λx.x x) (λx.x x)` is the minimal non-terminating expression.

Analysis:

- Smaller expressions like `x`, `λx.x`, `x x`, or `λx.x x` are all in normal form and terminate
- The omega combinator applies a self-applicative lambda `λx.x x` to itself
- When evaluated: `(λx.x x) (λx.x x)` → substitute `(λx.x x)` for `x` in `x x` → `(λx.x x) (λx.x x)` again, creating infinite recursion

This is minimal because it requires both a self-applicative lambda and applying that lambda to itself, which is exactly what the omega combinator provides.

2.13.5 Exercise 7: Step-by-Step Evaluation Trace

Tracing the evaluation of `((\m.\n. m n) (\f.\x. f (f x))) (\f.\x. f (f (f x)))` following the interpreter's exact steps.

Evaluation Trace:

1. `((\m.\n. m n) (\f.\x. f (f x))) (\f.\x. f (f (f x)))`
2. `((\Var1. (\f.\x. f (f x)) Var1)) (\f.\x. f (f (f x)))`
   - Substituting `(\f.\x. f (f x))` for `m` in `(\n. m n)`
   - The lambda variable `n` is renamed to `Var1` to avoid capture
   - Result: `(\Var1. (\f.\x. f (f x)) Var1)`
3. `((\f.\x. f (f x)) (\f.\x. f (f (f x))))`
   - Substituting `(\f.\x. f (f (f x)))` for `Var1` in `((\f.\x. f (f x)) Var1)`
   - Result: application of Church numeral 2 to Church numeral 3
4. `(\Var5. ((\f.\x. f (f (f x))) ((\f.\x. f (f (f x))) Var5)))`
   - Evaluating the application: `(\f.\x. f (f x)) (\f.\x. f (f (f x)))`
   - Substituting Church numeral 3 for `f` in `(\x. f (f x))`
   - The lambda variable `x` is renamed to `Var5` to avoid capture
   - Final result: partially applied function (not fully reduced to Church numeral 9)

Observations:

The interpreter uses call-by-name evaluation, so it doesn't fully reduce nested applications. The result shows a partially evaluated expression rather than the Church numeral 9, demonstrating that the evaluation strategy affects the final form.

2.13.6 Exercise 8: Recursive Call Trace

Tracing recursive calls to `evaluate()` and `substitute()` for `((\m.\n. m n) (\f.\x. f (f x))) (\f.\x. f x)` with indentation showing the call stack.

Recursive Trace:

```
37: eval ((((\m.(\n.(m n))) (\f.(\x.(f (f x))))) (\f.(\x.(f x))))
    39: eval (((\m.(\n.(m n))) (\f.(\x.(f (f x)))))
        39: eval (\m.(\n.(m n)))
    44: subst (\n.(m n)) [(\f.(\x.(f (f x))))/m]
        85: subst (m n) [Var1/n]
            85: subst m [Var1/n]
            85: subst n [Var1/n]
        85: subst (m Var1) [(\f.(\x.(f (f x))))/m]
            85: subst m [(\f.(\x.(f (f x))))/m]
            85: subst Var1 [(\f.(\x.(f (f x))))/m]
        39: eval (\Var1.((\f.(\x.(f (f x)))) Var1))
    44: subst ((\f.(\x.(f (f x)))) Var1) [(\f.(\x.(f x)))/Var1]
        85: subst (\f.(\x.(f (f x)))) [(\f.(\x.(f x)))/Var1]
            85: subst (\x.(f (f x))) [Var2/f]
                85: subst (f (f x)) [Var3/x]
                    85: subst f [Var3/x]
                    85: subst (f x) [Var3/x]
                        85: subst f [Var3/x]
                        85: subst x [Var3/x]
                85: subst (f (f Var3)) [Var2/f]
                    85: subst f [Var2/f]
                    85: subst (f Var3) [Var2/f]
                        85: subst f [Var2/f]
                        85: subst Var3 [Var2/f]
            85: subst (\Var3.(Var2 (Var2 Var3))) [(\f.(\x.(f x)))/Var1]
                85: subst (Var2 (Var2 Var3)) [Var4/Var3]
                    85: subst Var2 [Var4/Var3]
                    85: subst (Var2 Var3) [Var4/Var3]
                        85: subst Var2 [Var4/Var3]
                        85: subst Var3 [Var4/Var3]
                85: subst (Var2 (Var2 Var4)) [(\f.(\x.(f x)))/Var1]
                    85: subst Var2 [(\f.(\x.(f x)))/Var1]
                    85: subst (Var2 Var4) [(\f.(\x.(f x)))/Var1]
                        85: subst Var2 [(\f.(\x.(f x)))/Var1]
                        85: subst Var4 [(\f.(\x.(f x)))/Var1]
        85: subst Var1 [(\f.(\x.(f x)))/Var1]
    39: eval ((\Var2.(\Var4.(Var2 (Var2 Var4)))) (\f.(\x.(f x))))
        39: eval (\Var2.(\Var4.(Var2 (Var2 Var4))))
    44: subst (\Var4.(Var2 (Var2 Var4))) [(\f.(\x.(f x)))/Var2]
        85: subst (Var2 (Var2 Var4)) [Var5/Var4]
            85: subst Var2 [Var5/Var4]
            85: subst (Var2 Var4) [Var5/Var4]
                85: subst Var2 [Var5/Var4]
                85: subst Var4 [Var5/Var4]
        85: subst (Var2 (Var2 Var5)) [(\f.(\x.(f x)))/Var2]
            85: subst Var2 [(\f.(\x.(f x)))/Var2]
            85: subst (Var2 Var5) [(\f.(\x.(f x)))/Var2]
                85: subst Var2 [(\f.(\x.(f x)))/Var2]
                85: subst Var5 [(\f.(\x.(f x)))/Var2]
        39: eval (\Var5.((\f.(\x.(f x))) ((\f.(\x.(f x))) Var5)))
```

Observations:

- Line 37: Entry point to `evaluate()`

- Line 39: Recursive calls to `evaluate()` for left side and result evaluation
- Line 44: Calls to `substitute()` from `evaluate()` for beta-reduction
- Line 85: Recursive calls to `substitute()` for applications and nested substitutions
- Indentation shows the call stack depth, similar to the Tower of Hanoi trace
- The trace demonstrates the nested nature of capture-avoiding substitution, with multiple levels of variable renaming (Var1, Var2, Var3, Var4, Var5)

Question:

3 Essay

Throughout this course, a central theme has emerged: computation can be understood as the systematic transformation of symbolic expressions according to formal rules. This essay synthesizes the key concepts explored across the semester, showing how abstract rewriting systems, lambda calculus, formal proofs, and interpreter implementation connect to form a unified understanding of programming languages.

The course began with the MU puzzle, introducing the idea that computation is rule-based transformation. The puzzle's unsolvability demonstrated that not all goals are reachable within a formal system—a lesson that reappears throughout programming language theory. Abstract rewriting systems formalized this intuition, introducing termination, confluence, and normal forms. These properties determine whether a computation halts, produces consistent results, and what the final answer looks like.

Lambda calculus emerged as the theoretical core of the course. Unlike the concrete rewrite rules of the MU puzzle, lambda calculus provides a universal model of computation using only variables, abstraction, and application. Church encodings represent numbers through function composition, and beta-reduction alone suffices to compute any computable function. Fixed-point combinators extended this to recursion: the `fix` combinator enables self-reference without explicit naming, allowing definitions like factorial to unfold through substitution alone.

The interpreter project bridged theory and practice. Implementing a lambda calculus interpreter required translating beta-reduction into working code, with capture-avoiding substitution as the key challenge. The tracing exercises made evaluation visible, revealing the tree-like structure of computation—similar to the Tower of Hanoi traces. Both examples illustrated how recursive definitions decompose problems into smaller subproblems.

The Lean proof exercises connected logic to programming through the Curry-Howard correspondence, where propositions correspond to types and proofs correspond to programs. Induction proofs mirrored termination arguments: just as termination requires a decreasing measure, induction requires showing a property holds for successors given it holds for predecessors.

Several unifying principles emerge: substitution is fundamental to all computation; termination requires careful design to avoid unbounded self-reference; abstraction enables generality; and syntax and semantics, though distinct, are connected through interpretation. This course has shown that programming languages rest on precise mathematical foundations, enabling us not just to write code that works, but to understand *why* it works.

4 Evidence of Participation

Discord Seminar Posts:

- Programming Paradigms - Computerphile
- The Elegant Math Behind Machine Learning

5 Conclusion

Stepping back from the technical details, this course has reshaped how I think about programming and software engineering. Before taking CPSC-354, I viewed programming languages as tools—each with its own syntax to memorize and quirks to work around. Now I see them as designed artifacts built on mathematical foundations, with deliberate choices about evaluation strategy, type systems, and abstraction mechanisms that profoundly shape how we write and reason about code.

In the wider world of software engineering, these foundations matter more than they might first appear. Understanding termination and confluence helps when debugging infinite loops or reasoning about concurrent systems. Recognizing that parsing is grammar-driven explains why IDEs can catch syntax errors instantly and why language design involves careful attention to ambiguity. The Curry-Howard correspondence reveals that type systems are not just error-catching mechanisms but lightweight proof systems—a perspective that makes sense of why languages like Rust and TypeScript invest so heavily in expressive types. Even the lambda calculus, which can seem abstract, underlies the functional programming features now standard in JavaScript, Python, and Java.

The most valuable aspect of the course was the interpreter project. Building a working lambda calculus interpreter transformed abstract reduction rules into something tangible. Debugging capture-avoiding substitution forced me to truly understand alpha-renaming rather than just accepting it as a definition. The tracing exercises were similarly illuminating: watching the call stack grow and shrink made recursion concrete in a way that textbook examples never quite achieved. The Tower of Hanoi traces and the interpreter traces felt like two views of the same underlying phenomenon—recursive decomposition—and that connection was satisfying to discover.

The Lean proof exercises were initially frustrating but ultimately rewarding. Learning to think in terms of rewriting and induction felt like acquiring a new mental tool. However, the learning curve was steep, and more scaffolding in early exercises would help. I would suggest adding intermediate exercises that bridge the gap between simple rewrites and full induction proofs.

One improvement I would suggest is more explicit connections to mainstream languages. The course material is deep and rigorous, but occasionally felt disconnected from practical programming. Brief examples showing how, say, Python's closures relate to lambda abstraction, or how Haskell's laziness connects to evaluation strategy, would help ground the theory.

Overall, this course provided a foundation I expect to draw on throughout my career—not in daily coding, perhaps, but whenever I need to think carefully about what a program means and why it behaves as it does.

References

[NNG] Lean Community, Natural Number Game, Lean Game Server, 2025.

[PL2025] Alexander Kurz, Principles of Programming Languages (CPSC 354), Chapman University, 2025.